ASSISTANT RESPONSE

Μοντέλο : qwen3.5:397b

Ημερομηνία εξαγωγής: 7/4/2026, 9:33:14 π.μ.

Prompt Χρήστη

Λύσε και ερμήνευσε ένα πρόβλημα ιδιοτιμών και ιδιοδιανυσμάτων για πίνακα 2x2. Θέλω πλήρη βήματα, χαρακτηριστικό πολυώνυμο, έλεγχο λύσεων, γεωμετρική ερμηνεία και σύντομη αναφορά στη σύνδεση με διαγωνιοποίηση.

Απάντηση Assistant

Πρόβλημα Ιδιοτιμών και Ιδιοδιανυσμάτων για Πίνακα 2×2

1. Διατύπωση Προβλήματος

Δίνεται ο πίνακας:

$$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$$

Ζητούμενα:

1. Να βρεθούν οι **ιδιοτιμές** λ
2. Να βρεθούν τα αντίστοιχα **ιδιοδιανύσματα** $\vec{v}$
3. Να γίνει **έλεγχος** των λύσεων
4. Να δοθεί **γεωμετρική ερμηνεία**
5. Να αναφερθεί η σύνδεση με **διαγωνιοποίηση**

---

2. Θεωρητικό Υπόβαθρο

2.1 Ορισμοί

Όρος	Ορισμός
Ιδιοτιμή λ	Βαθμωτό μέγεθος για το οποίο υπάρχει μη μηδενικό $\vec{v}$ τέτοιο ώστε $A\vec{v} = \lambda\vec{v}$
Ιδιοδιάνυσμα $\vec{v}$	Μη μηδενικό διάνυσμα που ικανοποιεί $A\vec{v} = \lambda\vec{v}$
Χαρακτηριστική εξίσωση	$\det(A - \lambda I) = 0$

2.2 Βασική Εξίσωση

$$\boxed{A\vec{v} = \lambda\vec{v} \quad \Leftrightarrow \quad (A - \lambda I)\vec{v} = \vec{0}}$$

---

3. Υπολογισμός Ιδιοτιμών

3.1 Χαρακτηριστικός Πίνακας

$$A - \lambda I = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{pmatrix}$$

3.2 Χαρακτηριστικό Πολυώνυμο

$$\begin{aligned} p(\lambda) &= \det(A - \lambda I) \\ &= \begin{vmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{vmatrix} \\ &= (4-\lambda)(3-\lambda) - (1)(2) \\ &= 12 - 4\lambda - 3\lambda + \lambda^2 - 2 \\ &= \lambda^2 - 7\lambda + 10 \end{aligned}$$

$$\boxed{p(\lambda) = \lambda^2 - 7\lambda + 10}$$

3.3 Ρίζες του Χαρακτηριστικού Πολυωνύμου

$$\lambda^2 - 7\lambda + 10 = 0$$

Λύσεις της δευτεροβάθμιας εξίσωσης:

$$\lambda_{1,2} = \frac{7 \pm \sqrt{49-40}}{2} = \frac{7 \pm \sqrt{9}}{2} = \frac{7 \pm 3}{2}$$

$$\boxed{\lambda_1 = 5} \quad \text{και} \quad \boxed{\lambda_2 = 2}$$

3.4 Έλεγχος με Ίχνη και Ορίζουσα

Ιδιότητα	Τύπος	Υπολογισμός	Αποτέλεσμα
Ίχνος	$\mathrm{tr}(A) = \lambda_1 + \lambda_2$	$4 + 3 = 7$	$5 + 2 = 7$ ✓
Ορίζουσα	$\det(A) = \lambda_1 \cdot \lambda_2$	$4 \cdot 3 - 1 \cdot 2 = 10$	$5 \cdot 2 = 10$ ✓

---

4. Υπολογισμός Ιδιοδιανυσμάτων

4.1 Ιδιοδιάνυσμα για $\lambda_1 = 5$

Λύνουμε το σύστημα $(A - 5I)\vec{v}_1 = \vec{0}$:

$$A - 5I = \begin{pmatrix} 4-5 & 1 \\ 2 & 3-5 \end{pmatrix} = \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix}$$

Το σύστημα γίνεται:

$$\begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Από την πρώτη εξίσωση: $-x + y = 0 \Rightarrow y = x$

Επομένως:

$$\vec{v}_1 = \begin{pmatrix} x \\ x \end{pmatrix} = x \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Επιλέγουμε $x = 1$ για κανονικοποίηση:

$$\boxed{\vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}}$$

4.2 Ιδιοδιάνυσμα για $\lambda_2 = 2$

Λύνουμε το σύστημα $(A - 2I)\vec{v}_2 = \vec{0}$:

$$A - 2I = \begin{pmatrix} 4-2 & 1 \\ 2 & 3-2 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix}$$

Το σύστημα γίνεται:

$$\begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Από την πρώτη εξίσωση: $2x + y = 0 \Rightarrow y = -2x$

Επομένως:

$$\vec{v}_2 = \begin{pmatrix} x \\ -2x \end{pmatrix} = x \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Επιλέγουμε $x = 1$ για κανονικοποίηση:

$$\boxed{\vec{v}_2 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}}$$

5. Έλεγχος Λύσεων

5.1 Έλεγχος για $\lambda_1 = 5, \vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

$$A\vec{v}_1 = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4+1 \\ 2+3 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

$$\lambda_1 \vec{v}_1 = 5 \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

$$\boxed{A\vec{v}_1 = \lambda_1 \vec{v}_1} \quad \checkmark$$

5.2 Έλεγχος για $\lambda_2 = 2, \vec{v}_2 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$

$$A\vec{v}_2 = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 4-2 \\ 2-6 \end{pmatrix} = \begin{pmatrix} 2 \\ -4 \end{pmatrix}$$

$$\lambda_2 \vec{v}_2 = 2 \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 2 \\ -4 \end{pmatrix}$$

$$\boxed{A\vec{v}_2 = \lambda_2 \vec{v}_2} \quad \checkmark$$

---

6. Γεωμετρική Ερμηνεία

6.1 Φυσικό Νόημα

Ιδιοτιμή	Ιδιοδιάνυσμα	Ερμηνεία
$\lambda_1 = 5$	$\vec{v}_1 = (1,1)^T$	Κατά τη διεύθυνση αυτή, ο μετασχηματισμός **επιμηκύνει** κατά παράγοντα 5
$\lambda_2 = 2$	$\vec{v}_2 = (1,-2)^T$	Κατά τη διεύθυνση αυτή, ο μετασχηματισμός **επιμηκύνει** κατά παράγοντα 2

6.2 Οπτική Παράσταση

- Τα ιδιοδιανύσματα ορίζουν **αναλλοίωτες ευθείες** μέσω της αρχής των αξόνων
- Οποιοδήποτε διάνυσμα πάνω σε αυτές τις ευθείες παραμένει στην ίδια ευθεία μετά τον μετασχηματισμό
- Μόνο το **μέτρο** αλλάζει (κατά τον παράγοντα λ)

6.3 Γενικός Μετασχηματισμός

Για τυχαίο διάνυσμα $\vec{u} = c_1\vec{v}_1 + c_2\vec{v}_2$:

$$A\vec{u} = c_1\lambda_1\vec{v}_1 + c_2\lambda_2\vec{v}_2$$

---

7. Σύνδεση με Διαγωνιοποίηση

7.1 Πίνακας Ιδιοδιανυσμάτων

$$P = (\vec{v}_1 \quad \vec{v}_2) = \begin{pmatrix} 1 & 1 \\ 1 & -2 \end{pmatrix}$$

7.2 Διαγώνιος Πίνακας Ιδιοτιμών

$$D = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} = \begin{pmatrix} 5 & 0 \\ 0 & 2 \end{pmatrix}$$

7.3 Διαγωνιοποίηση

$$\boxed{A = PDP^{-1}}$$

όπου:

$$P^{-1} = \frac{1}{\det(P)} \begin{pmatrix} -2 & -1 \\ -1 & 1 \end{pmatrix} = \frac{1}{-3} \begin{pmatrix} -2 & -1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & -1/3 \end{pmatrix}$$

7.4 Εφαρμογές Διαγωνιοποίησης

Εφαρμογή	Περιγραφή
Δυνάμεις πίνακα	$A^n = PD^nP^{-1}$ λ λ μ
Εκθετική πίνακα	$e^{At} = Pe^{Dt}P^{-1}$ Δ
Ανάλυση συστημάτων	Αποσύζευξη γραμμικών συστημάτων
Κβαντική μηχανική	Ιδιοτιμές = μετρήσιμα μεγέθη

---

8. Οπτικοποίηση με Python / Matplotlib

PYTHON

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
""" eigenvalue_problem_2x2.py Visualization of eigenvalues and eigenvectors for a 2x2
matrix. """

import numpy as np
import matplotlib.pyplot as plt

# ==============================================================================
# Problem Definition
# ==============================================================================
```

```python
class EigenvalueProblem:
    """Class for solving and visualizing 2x2 eigenvalue problem."""

    def __init__(self, A):
        """Initialize with 2x2 matrix A."""
        self.A = np.array(A, dtype=float)
        self.eigenvalues, self.eigenvectors = np.linalg.eig(self.A)

    def characteristic_polynomial(self, lambda_val):
        """Evaluate characteristic polynomial p(lambda) = det(A - lambda*I)."""
        return np.linalg.det(self.A - lambda_val * np.eye(2))

    def verify_eigenpair(self, eigenvalue, eigenvector):
        """Verify that Av = lambda*v."""
        Av = self.A @ eigenvector
        lambda_v = eigenvalue * eigenvector
        return np.allclose(Av, lambda_v)

    def get_summary(self):
        """Return summary of eigenvalues and eigenvectors."""
        summary = []
        for i in range(2):
            v = self.eigenvectors[:, i]
            v_normalized = v / np.linalg.norm(v)
            summary.append({
                'index': i + 1,
                'eigenvalue': self.eigenvalues[i],
                'eigenvector': v,
                'eigenvector_normalized': v_normalized,
                'verified': self.verify_eigenpair(self.eigenvalues[i], v)
            })
        return summary


# ============================================================================
# Visualization Functions
# ============================================================================

def plot_eigenvectors_transformation(problem, ax):
    """ Plot eigenvectors before and after transformation. Parameters ----------
problem : EigenvalueProblem Problem instance ax : matplotlib.axes.Axes Plot axis """
    A = problem.A
    eigenvalues = problem.eigenvalues
    eigenvectors = problem.eigenvectors

    # Create grid of points
    x = np.linspace(-2, 2, 20)
```

```python
y = np.linspace(-2, 2, 20)
X, Y = np.meshgrid(x, y)
points = np.vstack([X.ravel(), Y.ravel()])

# Transform points
transformed = A @ points
XT = transformed[0, :].reshape(X.shape)
YT = transformed[1, :].reshape(Y.shape)

# Plot original grid (light)
ax.grid(True, alpha=0.3, linestyle='-', linewidth=0.5)

# Plot eigenvectors (original)
colors = ['red', 'blue']
for i in range(2):
    v = eigenvectors[:, i]
    v_norm = v / np.linalg.norm(v)

    # Original eigenvector
    ax.arrow(0, 0, v_norm[0], v_norm[1],
             head_width=0.15, head_length=0.2,
             fc=colors[i], ec=colors[i],
             linewidth=3, alpha=0.8, length_includes_head=True)

    # Transformed eigenvector (scaled by eigenvalue)
    v_transformed = eigenvalues[i] * v_norm
    ax.arrow(0, 0, v_transformed[0], v_transformed[1],
             head_width=0.15, head_length=0.2,
             fc=colors[i], ec=colors[i],
             linewidth=3, alpha=0.4, linestyle='--',
             length_includes_head=True)

    # Labels
    ax.text(v_norm[0]*1.1, v_norm[1]*1.1,
            'v%d' % (i+1), fontsize=12, fontweight='bold',
            color=colors[i])
    ax.text(v_transformed[0]*1.1, v_transformed[1]*1.1,
            'Av%d = %.1f*v%d' % (i+1, abs(eigenvalues[i]), i+1),
            fontsize=9, color=colors[i], alpha=0.7)

# Plot some sample vectors and their transformations
sample_angles = np.linspace(0, 2*np.pi, 8, endpoint=False)
for angle in sample_angles:
    u = np.array([np.cos(angle), np.sin(angle)])
    u_trans = A @ u

    # Original vector
```

```
        ax.arrow(0, 0, u[0]*0.5, u[1]*0.5,
                 head_width=0.08, head_length=0.1,
                 fc='gray', ec='gray',
                 linewidth=1.5, alpha=0.5, length_includes_head=True)

        # Transformed vector
        ax.arrow(0, 0, u_trans[0]*0.5/np.linalg.norm(u_trans),
                 u_trans[1]*0.5/np.linalg.norm(u_trans),
                 head_width=0.08, head_length=0.1,
                 fc='orange', ec='orange',
                 linewidth=1.5, alpha=0.5, length_includes_head=True)

    # Axis settings
    ax.set_xlim(-2.5, 2.5)
    ax.set_ylim(-2.5, 2.5)
    ax.set_aspect('equal')
    ax.set_xlabel('x', fontsize=11)
    ax.set_ylabel('y', fontsize=11)
    ax.set_title('Eigenvectors and Transformation',
                 fontsize=12, fontweight='bold')
    ax.axhline(y=0, color='k', linewidth=0.8, alpha=0.5)
    ax.axvline(x=0, color='k', linewidth=0.8, alpha=0.5)

    # Legend
    from matplotlib.patches import Patch
    legend_elements = [
        Patch(facecolor='red', alpha=0.8, label='Eigenvector v1'),
        Patch(facecolor='red', alpha=0.4, label='Av1 (scaled)'),
        Patch(facecolor='blue', alpha=0.8, label='Eigenvector v2'),
        Patch(facecolor='blue', alpha=0.4, label='Av2 (scaled)'),
        Patch(facecolor='gray', alpha=0.5, label='Other vectors'),
        Patch(facecolor='orange', alpha=0.5, label='Transformed')
    ]
    ax.legend(handles=legend_elements, loc='upper right', fontsize=8)


def plot_characteristic_polynomial(problem, ax):
    """ Plot characteristic polynomial p(lambda). Parameters ---------- problem :
EigenvalueProblem Problem instance ax : matplotlib.axes.Axes Plot axis """
    # Lambda range
    lambda_vals = np.linspace(-1, 8, 500)
    p_vals = [problem.characteristic_polynomial(l) for l in lambda_vals]

    # Plot polynomial
    ax.plot(lambda_vals, p_vals, 'm-', linewidth=2.5,
            label='p(lambda) = det(A - lambda*I)')
```

```
    # Mark eigenvalues
    eigenvalues = problem.eigenvalues
    for i, lam in enumerate(eigenvalues):
        ax.plot(lam, 0, 'go', markersize=12, zorder=10)
        ax.annotate('lambda%d = %.1f' % (i+1, lam),
                    xy=(lam, 0), xytext=(lam, 5),
                    fontsize=10, fontweight='bold', color='darkgreen',
                    arrowprops=dict(arrowstyle='->', color='darkgreen',
                                    linewidth=1.5))

    # Axis lines
    ax.axhline(y=0, color='k', linewidth=1.5, alpha=0.8)
    ax.axvline(x=0, color='k', linewidth=1.5, alpha=0.8)

    # Settings
    ax.set_xlabel('lambda', fontsize=11)
    ax.set_ylabel('p(lambda)', fontsize=11)
    ax.set_title('Characteristic Polynomial', fontsize=12, fontweight='bold')
    ax.grid(True, alpha=0.4, linestyle='--')
    ax.legend(loc='upper right', fontsize=9)
    ax.set_xlim(-1, 8)
    ax.set_ylim(min(p_vals)*1.1, max(p_vals)*1.1)


def plot_eigenvalue_scaling(problem, ax):
    """ Show scaling effect of eigenvalues on eigenvectors. Parameters ----------
problem : EigenvalueProblem Problem instance ax : matplotlib.axes.Axes Plot axis """
    eigenvalues = problem.eigenvalues
    eigenvectors = problem.eigenvectors

    # Create bar chart of eigenvalues
    colors = ['red', 'blue']
    bars = ax.bar([1, 2], eigenvalues, color=colors, alpha=0.7,
                  edgecolor='black', linewidth=2)

    # Add value labels
    for i, (bar, lam) in enumerate(zip(bars, eigenvalues)):
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.1,
                'lambda%d = %.1f' % (i+1, lam),
                ha='center', va='bottom', fontsize=11, fontweight='bold')

    # Reference line at y=1
    ax.axhline(y=1, color='green', linestyle='--', linewidth=2,
               alpha=0.6, label='No scaling (lambda=1)')

    # Settings
```

```python
    ax.set_xlabel('Eigenvalue Index', fontsize=11)
    ax.set_ylabel('Eigenvalue Magnitude', fontsize=11)
    ax.set_title('Eigenvalue Scaling Factors', fontsize=12, fontweight='bold')
    ax.set_xticks([1, 2])
    ax.set_xticklabels(['1', '2'])
    ax.grid(True, alpha=0.3, linestyle='--', axis='y')
    ax.legend(loc='upper right', fontsize=9)
    ax.set_ylim(0, max(eigenvalues)*1.3)


# ==============================================================================
# Main Program
# ==============================================================================

if __name__ == "__main__":
    # Define the matrix from the problem
    A = [[4, 1], [2, 3]]

    # Create problem instance
    problem = EigenvalueProblem(A)

    # Print results to console
    print("=" * 70)
    print("EIGENVALUE PROBLEM SOLUTION - 2x2 MATRIX")
    print("=" * 70)
    print("\nMATRIX A:")
    print(" | %.0f %.0f |" % (A[0][0], A[0][1]))
    print(" | %.0f %.0f |" % (A[1][0], A[1][1]))

    print("\nEIGENVALUES:")
    summary = problem.get_summary()
    for item in summary:
        print(" lambda%d = %.4f" % (item['index'], item['eigenvalue']))
        print(" Eigenvector v%d = [%.4f, %.4f]^T" % (
            item['index'],
            item['eigenvector_normalized'][0],
            item['eigenvector_normalized'][1]))
        print(" Verified: %s" % item['verified'])

    print("\nTRACE CHECK:")
    print(" tr(A) = %.1f, sum(eigenvalues) = %.1f" % (
        np.trace(np.array(A)), np.sum(problem.eigenvalues)))

    print("\nDETERMINANT CHECK:")
    print(" det(A) = %.1f, product(eigenvalues) = %.1f" % (
        np.linalg.det(np.array(A)), np.prod(problem.eigenvalues)))
```

```python
print("\n" + "=" * 70)

# Create plots
fig = plt.figure(figsize=(14, 10))
fig.suptitle('Eigenvalue Problem: 2x2 Matrix Analysis',
             fontsize=14, fontweight='bold', y=0.98)

# Plot 1: Eigenvectors and transformation
ax1 = fig.add_subplot(2, 2, 1)
plot_eigenvectors_transformation(problem, ax1)

# Plot 2: Characteristic polynomial
ax2 = fig.add_subplot(2, 2, 2)
plot_characteristic_polynomial(problem, ax2)

# Plot 3: Eigenvalue scaling
ax3 = fig.add_subplot(2, 2, 3)
plot_eigenvalue_scaling(problem, ax3)

# Plot 4: Matrix visualization
ax4 = fig.add_subplot(2, 2, 4)
im = ax4.imshow(np.array(A), cmap='viridis', aspect='auto')
ax4.set_title('Matrix A Visualization', fontsize=12, fontweight='bold')
ax4.set_xticks([0, 1])
ax4.set_yticks([0, 1])
ax4.set_xticklabels(['1', '2'])
ax4.set_yticklabels(['1', '2'])

# Add value annotations
for i in range(2):
    for j in range(2):
        ax4.text(j, i, '%.0f' % A[i][j],
                 ha='center', va='center', fontsize=14,
                 fontweight='bold', color='white')

plt.colorbar(im, ax=ax4, label='Value')

# Final settings
plt.tight_layout(rect=[0, 0.02, 1, 0.96])

# Save and display
OUTPUT_PATH = "eigenvalue_analysis_2x2.png"
plt.savefig(OUTPUT_PATH, dpi=300, bbox_inches='tight')
print("\nPlot saved: %s" % OUTPUT_PATH)

plt.show()
```

9. Συνοπτικός Πίνακας Αποτελεσμάτων

Στοιχείο	Τιμή	Επαλήθευση
Πίνακας	$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$	—
Χαρακτηριστικό πολυώνυμο	$p(\lambda) = \lambda^2 - 7\lambda + 10$	—
Ιδιοτιμή λ_1	5	✓
Ιδιοτιμή λ_2	2	✓
Ιδιοδιάνυσμα $\vec{v}_1$	$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	$A\vec{v}_1 = 5\vec{v}_1$ ✓
Ιδιοδιάνυσμα $\vec{v}_2$	$\begin{pmatrix} 1 \\ -2 \end{pmatrix}$	$A\vec{v}_2 = 2\vec{v}_2$ ✓
Ίχνος	$\mathrm{tr}(A) = 7 = \lambda_1 + \lambda_2$	✓
Ορίζουσα	$\det(A) = 10 = \lambda_1 \cdot \lambda_2$	✓

---

10. Προαπαιτούμενα

Απαίτηση	Εντολή
Python	Έκδοση 3.8+
NumPy	`pip install numpy`
Matplotlib	`pip install matplotlib`
Εκτέλεση	`python eigenvalue_problem_2x2.py`
Έξοδος	`eigenvalue_analysis_2x2.png` + console output

11. Συμπεράσματα

1. **Οι ιδιοτιμές είναι πραγματικές και διακριτές** → ο πίνακας είναι διαγωνιοποιήσιμος
1. **Τα ιδιοδιανύσματα είναι γραμμικά ανεξάρτητα** → σχηματίζουν βάση του $\mathbb{R}^2$
1. **Γεωμετρικά**: Ο μετασχηματισμός επιμηκύνει κατά 5 στη διεύθυνση $(1, 1)$ και κατά 2 στη διεύθυνση $(1, -2)$
1. **Η διαγωνιοποίηση** $A = PDP^{-1}$ απλοποιεί υπολογισμούς δυνάμεων και εκθετικών πινάκων